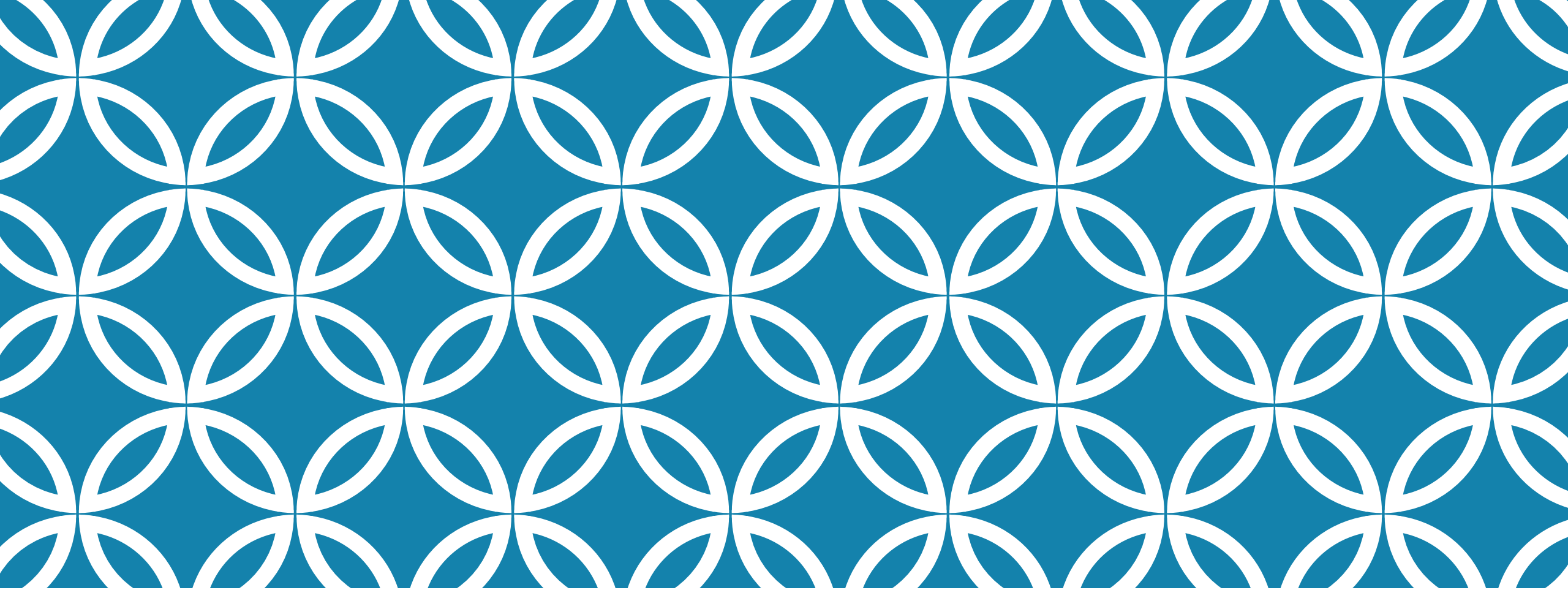

SEC4_DATA SCIENCE

Eng.Omnya Negm

DASK

DASK

Dask is a **parallel computing library** in Python that enables **scaling computations** from a single machine to a distributed cluster. It is designed to handle large datasets and optimize computations that go beyond memory limitations.

Why Use Dask?

- **Handles large datasets** that don't fit into memory
- **Parallelizes computations** to improve speed
- **Integrates with Pandas, NumPy, and SciPy** for easy adoption
- **Lazy execution** to optimize resource usage

Scenario	Use Dask?
Small dataset	❌ No need, Pandas/NumPy is fine
Large dataset (> RAM)	✅ Yes, use Dask DataFrame
Large matrix computations	✅ Yes, use Dask Array
Many function calls (parallel execution)	✅ Yes, use Dask Delayed
Distributed computing	✅ Yes, use Dask Distributed

DASK

Dask Component	Equivalent Library	Use Case
`dask.array`	NumPy	Parallel array computations
`dask.dataframe`	Pandas	Large-scale data processing
`dask.delayed`	Python functions	Lazy evaluation of functions
`dask.bag`	Lists, JSON	Parallel processing of unstructured data
`dask.distributed`	-	Distributed computing across multiple machines

DASK CORE COMPONENT

w folder > Untitled-1.py

```python
import dask
from dask import delayed
import time

def slow_function(x):
    time.sleep(2)
    return x * 2

# Convert function calls into lazy tasks
delayed_results = [delayed(slow_function)(i) for i in range(5)]

# Execute in parallel
results = dask.compute(*delayed_results)
print(results)
```

DASK

Dask works by **breaking large computations into smaller tasks** and processing them in parallel.

Example: Parallel Function Execution with Dask Delayed

DASK

Dask Arrays allow you to work with large datasets that **don't fit in memory.**

Benefits of Dask Arrays

✓ Handles **larger-than-memory** arrays

✓ Computes **faster using parallelism**

Example 2: Creating and Processing Large Arrays

```
import dask.array as da

# Create a large array (10,000 x 10,000), but only load small chunks into memory
large_array = da.random.random((10000, 10000), chunks=(1000, 1000))

# Perform computations in parallel
mean_value = large_array.mean().compute()  # Compute the mean
print("Mean Value:", mean_value)
```

DASK

Dask DataFrames are useful for **handling large CSV files** that don't fit in RAM.

Example3 : Loading and Processing a Large CSV

Why use Dask DataFrames?
✓ Works **with large datasets**
✓ **Same API as Pandas,** making it easy to learn

```
import dask.dataframe as dd

# Load a large CSV file in chunks (lazy loading)
df = dd.read_csv("large_dataset.csv")

# Perform operations
df_filtered = df[df["column_name"] > 100]  # Filter rows
df_filtered["new_column"] = df_filtered["existing_column"] * 2  # Create a new column

# Compute results (Dask only executes when compute() is called)
result = df_filtered.compute()
print(result.head())
```

DASK

Dask Bags work for **semi-structured data** like logs, JSON files, or text.

Why use Dask Bags?

✓ Works **on unstructured or streaming data**

✓ Can **process JSON, logs, or text files in parallel**

Example 4: Processing a List of JSON Records

```python
import dask.bag as db

# Create a Dask Bag from a list
data = db.from_sequence([{"name": "Alice", "age": 25}, {"name": "Bob", "age": 30}])

# Extract names
names = data.pluck("name").compute()
print(names)  # Output: ['Alice', 'Bob']
```

DASK

Dask can scale beyond a single machine by distributing tasks across **multiple CPUs or clusters.**

Example 5: Running Dask on Multiple Cores

```
from dask.distributed import Client

# Start a Dask cluster
client = Client()

# Print cluster info
print(client)
```

Concept	Dask Feature Used	Equivalent in Python
Parallel function execution	`dask.delayed`	Normal Python functions
Handling large arrays	`dask.array`	NumPy
Processing large CSV files	`dask.dataframe`	Pandas
Handling unstructured data	`dask.bag`	Python lists, JSON
Running on multiple machines	`dask.distributed`	Multiprocessing

DASK

TASK

Task 1: Load and Filter Data
You have a large CSV file containing sales data with multiple columns, including "Sales" and "Date". Your task is to use Dask to efficiently load this dataset without consuming too much memory. Once the dataset is loaded, filter out all rows where the "Sales" value is greater than 1000. After filtering, display the first 10 rows of the resulting dataset to verify that the operation was successful.

Task 2: Compute Average Sales by Category
Given a CSV file that contains product sales information, where each row includes a "Category" column and a "Sales" column, your task is to use Dask to compute the average sales for each product category. Since Dask processes data lazily, ensure that you explicitly compute the results before displaying them. Finally, convert the output into a Pandas DataFrame and display the average sales for each category.

Task 3: Find the Top 5 Most Profitable Products
You are provided with a CSV file that records product details, including "Product" and "Profit" columns. Your task is to use Dask to find the top 5 products with the highest profit. First, sort the dataset in descending order based on the "Profit" column to identify the most profitable products. After sorting, extract the top 5 entries and display both the product names and their corresponding profit values.

MATPLOTLIB

MATPLOTLIB

Matplotlib is a **data visualization** library in Python used for **creating graphs, charts, and plots.** It is widely used in **data science** to visualize trends, distributions, and relationships in data.

MATPLOTLIB

```python
import matplotlib.pyplot as plt

# Data
x = [1, 2, 3, 4, 5]
y = [10, 15, 7, 12, 20]

# Create a line plot
plt.plot(x, y, marker='o', linestyle='-', color='b', label="Sales")

# Labels and Title
plt.xlabel("Days")
plt.ylabel("Sales")
plt.title("Sales Over Time")
plt.legend()

# Show the plot
plt.show()
```

Example1: Line Plot

Key Features Used:

✓ plot() → Creates the line plot

✓ marker='o' → Adds markers at data points

✓ xlabel(), ylabel(), title() → Add labels and title

MATPLOTLIB

```
import matplotlib as plt
# Data
categories = ["A", "B", "C", "D"]
values = [40, 70, 30, 90]

# Create a bar chart
plt.bar(categories, values, color=['red', 'blue', 'green', 'purple'])

# Labels and Title
plt.xlabel("Categories")
plt.ylabel("Values")
plt.title("Category Comparison")

plt.show()
```

Example2: Bar Chart

Why use bar charts?

✓ Compare categories easily

✓ Can show **grouped or stacked bars**

MATPLOTLIB

```python
import numpy as np
import matplotlib as plt

# Generate random data
x = np.random.rand(50)
y = np.random.rand(50)

# Scatter plot
plt.scatter(x, y, color='green', alpha=0.6)

# Labels and Title
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Scatter Plot Example")

plt.show()
```

Example3: Scatter Plot

Why use scatter plots?

✓ Useful for **finding correlations**

✓ Shows the **spread of data points**

MATPLOTLIB

```python
import numpy as np
import matplotlib as plt

# Generate random data
data = np.random.randn(1000)

# Create a histogram
plt.hist(data, bins=30, color='blue', edgecolor='black')

# Labels and Title
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.title("Histogram Example")

plt.show()
```

Example 4: Histogram

Why use histograms?

✓ Shows **distribution of data**

✓ Helps identify **outliers and trends**

MATPLOTLIB

```python
import numpy as np
import matplotlib as plt

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

# Line Plot
axes[0].plot(x, y, marker='o', color='b')
axes[0].set_title("Line Plot")

# Bar Chart
axes[1].bar(categories, values, color='red')
axes[1].set_title("Bar Chart")

plt.tight_layout()
plt.show()
```

Example 5: Multiple subplots

Why use subplots?

✓ Helps **compare different visualizations**

✓ Saves space instead of creating multiple plots

MATPLOTLIB

Example 6: Customizing plots Customizations Used:

✓ plt.style.use() → Changes the entire plot's style

✓ linewidth=2, markersize=8 → Adjusts line and marker size

```
import numpy as np
import matplotlib as plt

plt.style.use('seaborn-darkgrid')  # Apply a custom style

# Create a line plot with customizations
plt.plot(x, y, marker='o', linestyle='-', color='b', linewidth=2, markersize=8)

# Labels and Title
plt.xlabel("Days", fontsize=12)
plt.ylabel("Sales", fontsize=12)
plt.title("Sales Trend", fontsize=14)

plt.show()
```

Chart Type	Use Case
Line Plot	Show trends over time
Bar Chart	Compare categories
Scatter Plot	Find relationships between variables
Histogram	Show data distribution
Subplots	Compare multiple plots in one figure

SUMMERY MATPLOTLIB

SUMMERY MATPLOTLIB

How to Visualize a CSV File Using Matplotlib

CSV files contain structured data, and we can use **Matplotlib** along with **Pandas** to load and visualize them.

```
import pandas as pd
import matplotlib as plt

plt.style.use('seaborn-darkgrid')  # Apply a custom style
# Create a line plot with customizations
plt.plot(x, y, marker='o', linestyle='-', color='b', linewidth=2, markersize=8)
# Labels and Title
plt.xlabel("Days", fontsize=12)
plt.ylabel("Sales", fontsize=12)
plt.title("Sales Trend", fontsize=14)
plt.show()
# Load CSV
df = pd.read_csv("sales_data.csv")
# Group by category and sum sales
category_sales = df.groupby("Category")["Sales"].sum()
# Create a bar chart
plt.bar(category_sales.index, category_sales.values, color=['red', 'blue', 'green'])
# Customize the plot
plt.xlabel("Product Category")
plt.ylabel("Total Sales")
plt.title("Sales by Category")
plt.show()
```

TASK

Task 1: Create a Line Chart to Show Temperature Changes
You have a dataset that records the temperature of a city over a week. Your task is to use Matplotlib to create a line chart where the x-axis represents the days of the week (Monday to Sunday) and the y-axis represents the temperature recorded each day. Ensure that the chart has appropriate axis labels, a title, and a grid to make the data easier to understand. Customize the line color and style for better visualization.

Task 2: Plot a Bar Chart to Compare Sales of Different Products
A store sells five different products, and you want to visualize their sales performance. Your task is to create a bar chart using Matplotlib where the x-axis represents the product names, and the y-axis represents the number of units sold. Each bar should have a different color to make the chart visually appealing. Add appropriate labels, a title, and display the exact sales numbers on top of each bar.

Task 3: Create a Pie Chart to Show the Percentage of Students in Different Courses
A university has students enrolled in four different courses: Computer Science, Business, Engineering, and Arts. Your task is to use Matplotlib to create a pie chart that represents the percentage of students in each course. Use different colors for each category and include a legend to show which color corresponds to each course. Also, display the percentage values inside the pie chart to make it more informative.